

# Software Requirements Specification (SRS)

## AeroSwarm: Multi-Agent Parallel Cloud IDE

Product & Engineering Architecture

March 17, 2026

### Contents

|          |                                                                   |          |
|----------|-------------------------------------------------------------------|----------|
| <b>1</b> | <b>Executive Summary &amp; Product Vision</b>                     | <b>1</b> |
| <b>2</b> | <b>Target Audience &amp; Use Cases</b>                            | <b>2</b> |
| <b>3</b> | <b>Functional Product Requirements (FR)</b>                       | <b>2</b> |
| 3.1      | FR1: The AI Orchestration Engine . . . . .                        | 2        |
| 3.2      | FR2: Workspace Isolation & Management . . . . .                   | 2        |
| 3.3      | FR3: The “Grid” Dashboard (User Interface) . . . . .              | 2        |
| 3.4      | FR4: Parallel Live Previews . . . . .                             | 2        |
| 3.5      | FR5: The “Janitor” Merge Protocol . . . . .                       | 3        |
| <b>4</b> | <b>Technical Architecture &amp; Infrastructure</b>                | <b>3</b> |
| 4.1      | Recommended Technology Stack . . . . .                            | 3        |
| 4.2      | The Execution Pipeline (System Data Flow) . . . . .               | 3        |
| <b>5</b> | <b>Security &amp; Safeguards</b>                                  | <b>4</b> |
| <b>6</b> | <b>Vendor Deliverables &amp; Milestones</b>                       | <b>4</b> |
| 6.1      | Phase 1: Core Engine & CLI (Weeks 1-4) . . . . .                  | 4        |
| 6.2      | Phase 2: Web Interface & Real-Time Sync (Weeks 5-8) . . . . .     | 4        |
| 6.3      | Phase 3: Proxy Previews & Merge Resolution (Weeks 9-12) . . . . . | 4        |

---

### 1 Executive Summary & Product Vision

**AeroSwarm** is a web-based, multi-agent Integrated Development Environment (IDE). Unlike traditional AI coding assistants (which operate serially in a single chat interface), AeroSwarm acts as a “Parallel Software Factory.”

It allows a single user (Project Manager/Lead Developer) to delegate high-level features to an Orchestrator AI. The Orchestrator breaks the feature into sub-tasks and spins up multiple specialized AI Worker Agents. These agents work concurrently in completely isolated physical workspaces (Git Worktrees within Docker containers) on the same codebase. The user can monitor their terminals, view live application previews via dynamic subdomains, and merge the code through a visual conflict-resolution interface.

## 2 Target Audience & Use Cases

- **Solo Founders / Indie Hackers:** Rapidly prototyping full-stack applications by having agents build the frontend, backend, and database schema simultaneously.
- **Engineering Managers:** Automating large-scale refactoring, unit test generation, and documentation across a massive codebase without tying up human developers.
- **QA & Testing Teams:** Running parallel branches to test different UI implementations or dependency upgrades to see which one performs best in a live preview.

## 3 Functional Product Requirements (FR)

### 3.1 FR1: The AI Orchestration Engine

- **Task Decomposition:** The system must accept a primary prompt (e.g., "Build a Stripe checkout flow") and use a Manager LLM to generate a structured JSON list of independent sub-tasks.
- **Agent Assignment:** The system must provision a unique AI Worker for each sub-task, assigning it a specific scope (e.g., "You may only edit /src/api").

### 3.2 FR2: Workspace Isolation & Management

- **Git Worktrees:** Each agent must operate in a dedicated Git Worktree linked to the main repository to prevent file-locking and collision.
- **Dependency Management:** The system must automatically install dependencies (npm install, pip install) when a workspace is initialized.

### 3.3 FR3: The "Grid" Dashboard (User Interface)

- **Multi-View IDE:** A browser-based dashboard displaying a grid of active agents.
- **Live Terminals:** Each grid item must feature an integrated terminal emulator streaming the agent's standard output in real-time.
- **Virtual File System (VFS):** An integrated code editor allowing the user to browse and manually edit files inside any agent's specific worktree.

### 3.4 FR4: Parallel Live Previews

- **Dynamic Routing:** The platform must assign a unique port to each agent's dev server and expose it via a dynamic, secure URL.
- **Side-by-Side Viewing:** The UI must support a split screen to visually compare the live applications generated by different agents simultaneously.

### 3.5 FR5: The “Janitor” Merge Protocol

- **Automated Pre-flight:** Before an agent’s branch is marked “Ready to Merge,” the system must run automated linters and tests.
- **Visual Diff & Conflict Resolution:** The user must be presented with a standard 3-way Git merge UI to review code changes.
- **Auto-Cleanup:** Upon successful merge to the main branch, the agent’s Docker container and Git Worktree must be safely destroyed.

## 4 Technical Architecture & Infrastructure

To achieve true parallel execution without race conditions, the vendor must implement a highly scalable, containerized architecture.

### 4.1 Recommended Technology Stack

| Layer          | Recommended Technology      | Justification                                                       |
|----------------|-----------------------------|---------------------------------------------------------------------|
| Frontend       | Next.js, React, Tailwind UI | Fast, server-rendered UI with robust Web-Socket support.            |
| Backend API    | Python (FastAPI) or Node.js | High-performance async handling for orchestrating system commands.  |
| Agent Logic    | LangGraph or CrewAI         | Native support for stateful multi-agent workflows.                  |
| Infrastructure | Docker, Kubernetes (K8s)    | Essential for sandboxing agents into isolated runtime environments. |
| Networking     | Traefik or Nginx Proxy      | Required for dynamic subdomain routing to parallel container ports. |
| Database       | PostgreSQL + Redis          | Postgres for persistent state; Redis for real-time pub/sub logging. |

### 4.2 The Execution Pipeline (System Data Flow)

1. **Init:** User submits a prompt via the Next.js UI.
2. **Plan:** FastAPI backend calls the Manager LLM to break down tasks.
3. **Spawn:** FastAPI triggers the Docker daemon to spin up N containers. Each container mounts the main repository volume and executes `git worktree add`.
4. **Execute:** The LangGraph worker loops inside the container. It uses tools to read files, write code, and run shell commands.
5. **Stream:** The container’s output is captured and piped through Redis to the Next.js frontend via WebSockets.
6. **Preview:** The agent runs a dev server inside the container. Traefik dynamically maps the URL to that specific container’s IP and port.

## 5 Security & Safeguards

- **Network Isolation:** Agent containers must not have access to the internal VPC/Network beyond pulling public packages and communicating with the Orchestrator API.
- **Directory Sandboxing:** Agents must be strictly chroot jailed or restricted to their specific Git Worktree directory.
- **Secret Management:** API keys must be injected at runtime via secure environment variables.
- **Human-in-the-Loop (HITL):** No code generated by an agent can be pushed to the remote main branch without explicit approval from the human user.

## 6 Vendor Deliverables & Milestones

### 6.1 Phase 1: Core Engine & CLI (Weeks 1-4)

**Goal:** Prove the backend architecture works.

**Deliverables:** A backend that can successfully take a prompt, spawn two isolated Docker containers, create two Git worktrees, and have two LLMs write code simultaneously without file corruption.

### 6.2 Phase 2: Web Interface & Real-Time Sync (Weeks 5-8)

**Goal:** Bring the system to the browser.

**Deliverables:** The Next.js frontend featuring the Kanban grid, code editor integration, and real-time WebSocket terminal streaming.

### 6.3 Phase 3: Proxy Previews & Merge Resolution (Weeks 9-12)

**Goal:** Complete the “Cloud IDE” experience.

**Deliverables:** Implementation of Traefik/Nginx for dynamic live preview URLs, the visual Git merge UI, and the final CI/CD “Janitor” testing loop. Deployment scripts (Terraform/Helm charts) for cloud hosting.